عوالم البرمجة

أكاديمية عوالم البرمجة

Git & GitHub 101

من الصفر إلى أول Pull Request

.BEYOND CODE. BEYOND LIMITS

04

الوحدة 04

git add وcommit والسجل

اختر ما تريد حفظه، واحفظه برسالة واضحة، واقرأ التاريخ الذي صنعته.

فهم git add

Understanding git add

أهداف التعلم

- أن تشرح معنى التجهيز (staging).
- أن تجهّز كل شيء أو ملفات محددة فقط.
- أن تفرّق بين `git add .` و `git add -A` .
- أن تفحص ما جُهّز.
- أن تشرح لماذا يجب ألا تُحفظ الأسرار في commit أبدًا.

المتطلبات المسبقة

الوحدة 3 ومجلد `my-first-project` .

مقدمة

لا يحفظ Git كل تغيير تلقائيًا. أولًا تختار التغييرات التي تنتمي إلى اللقطة القادمة. هذا الاختيار يسمى التجهيز، والأمر `git add` هو أداته.

الشرح

التجهيز (Staging) يعني وضع التغييرات في منطقة التجهيز، وهي غرفة انتظار الـ commit القادم. الأمر `git add` لا ينشئ commit أبدًا، هو يحضّره فقط.

`git add index.html` يجهّز ملفًا واحدًا. و `git add index.html style.css` يجهّز ملفين. و `git add .` يجهّز كل تغيير في المجلد الحالي ومجلداته الفرعية: ملفات جديدة ومعدّلة ومحذوفة.

`git add -A` يجهّز كل التغييرات في المستودع كله أينما كنت داخله. ومن جذر المشروع يفعل `.` و `-A` الشيء نفسه، ويختلفان عند تنفيذهما داخل مجلد فرعي.

`git add -p` يتيح لك التجهيز قطعة قطعة. يعرض Git كل كتلة متغيرة (*hunk*) ويسأل `Stage this hunk [y,n,q,a,d,s,e,?]` . أجب `y` للموافقة و `n` للرفض و `q` للخروج.

لماذا التجهيز الانتقائي؟ إن أصلحت خطأً وبدأت ميزة جديدة، فـ commitان أوضح من واحد. الملفات المحددة أو `-p` تتيح لك فصل العمل.

الفحص. الأمر `git status` يعرض الملفات المجهّزة تحت «Changes to be committed». والأمر `git diff --staged` يعرض بالضبط ما جُهّز. وإن عدّلت ملفًا **بعد** تجهيزه فسيظهر في القائمتين: الـ commit يأخذ النسخة التي جهّزتها لا تعديلك الأحدث. نفّذ `git add` مرة أخرى لتضمّه.

الأسرار. لا تجهّز ملفات فيها كلمات سر أو مفاتيح API أو بيانات `.env` . بعد حفظها في commit تبقى في التاريخ. افحص `git status` قبل `git add .` .

في VS Code. افتح Source Control (`Ctrl+Shift+G`). علامة **+** بجانب الملف تجهّزه، وعلامة **+** بجانب **Changes** تجهّزها كلها (مثل `git add .`)، وعلامة **−** تلغي التجهيز.

تشبيه من الحياة

التجهيز كاختيار الأشياء لصورة. ترتّب الأغراض التي تريدها في الإطار أولًا، ثم يأتي ضغط الزر (الـ commit).

مخطط توضيحي

رسم توضيحي · Git Bash (ليس لقطة حقيقية)

```
$ git add index.html
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        style.css
```

الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

`git add index.html`

`git add index.html style.css`

`git add .`

`git add -A`

```
git add -p
git status
```

الخيارات: `.` المجلد الحالي؛ `-A` كل التغييرات في المستودع؛ `-p` اختيار الكتل بشكل تفاعلي. الشكل 1 يعرض `git status` بعد تجهيز ملف واحد.

أخطاء شائعة وحلولها

الظن أن git add يحفظ العمل.	لا يُحفظ شيء حتى تنفّذ commit.
استخدام git add . دون نظر.	نفّذ `git status` أولًا حتى لا يمرّ سر أو ملف غير مرغوب.
التعديل بعد التجهيز ثم commit دون add جديد.	نفّذ `git add` مرة أخرى، وإلا بقي التعديل الجديد خارج الـ commit.

تمرين عملي

في `my-first-project` جهّز `index.html` فقط. نفّذ `git status` ثم نفّذ `git diff --staged`.

النتيجة المتوقعة

يظهر `index.html` تحت «Changes to be committed» ويبقى الملفان الآخران غير متتبَّعين، كما في الشكل 1.

اختبر فهمك

1. هل ينشئ git add عملية commit؟

- ○ في المرة الأولى فقط
- ○ لا، هو يجهّز فقط
- ○ نعم

2. أي أمر يعرض ما جُهّز؟

- ○ git diff --staged
- ○ git push
- ○ git init

3. لماذا لا نحفظ ملف .env في commit؟

- ○ لأن Git لا يقرؤه
- ○ قد يحتوي أسرارًا تبقى في التاريخ
- ○ لأنه كبير

الإجابات

- لا، هو يجهّز فقط — التجهيز يحضّر التغييرات، والـ commit ينشئ اللقطة.
- git diff --staged — يقارن منطقة التجهيز بآخر commit.
- قد يحتوي أسرارًا تبقى في التاريخ — كل ما يُحفظ في commit يبقى في التاريخ.

الخلاصة

الأمر git add يجهّز التغييرات للـ commit القادم. جهّز ملفات محددة لتحصل على commits نظيفة، وافحص بـ git status وgit diff --staged، ولا تجهّز الأسرار أبدًا.

مصطلحات أساسية

منطقة التجهيز	غرفة انتظار الـ commit القادم
git add -A	تجهيز كل التغييرات في المستودع
Hunk	كتلة متغيرة من ملف

الدرس التالي

الدرس 4.2 — فهم git commit. تغييراتك مجهّزة. الآن تحفظها.

فهم git commit

Understanding git commit

أهداف التعلم

- أن تشرح ما هو الـ commit.
- أن تنشئ commit برسالة جيدة.
- أن تقرأ commit hash.
- أن تفرّق بين الرسائل الجيدة والسيئة.

المتطلبات المسبقة

الدرس 4.1.

مقدمة

الـ commit هو اللحظة التي تحفظ فيها لقطة في التاريخ. هذا قلب Git.

الشرح

الـ commit لقطة من التغييرات المجهّزة مع رسالة واسم صاحبها وتاريخ. يُخزَّن في تاريخك المحلي ويرتبط بالـ commit الذي قبله.

الأمر `git commit -m "Add homepage"` ينشئ commit بهذه الرسالة. الخيار `-m` يعني «رسالة». ضع الرسالة بين علامتي اقتباس.

يطبع Git سطرًا مثل `[main (root-commit) 3f2a9c1] Add homepage` . **main** هو الفرع، و**root-commit** يدل على أول commit في المستودع، و**3f2a9c1** بداية **commit hash** وهو معرّف فريد.

تبقى الـ commits على جهازك. لا شيء على GitHub حتى تنفّذ push (الوحدة 5).

الرسائل الجيدة تبدأ بفعل وتقول ما الذي تغيّر: `Add homepage` و `Fix navigation styling` و `Update README` . **الرسائل السيئة** لا تقول شيئًا: `changes` و `stuff` و `asdf` .

commits ذات معنى. يجب أن يمثل كل commit تغييرًا منطقيًا واحدًا يعمل. هذا يجعل التاريخ مقروءًا والتراجع عن الأخطاء سهلًا.

تشبيه من الحياة

الـ commit نقطة حفظ في لعبة فيديو مع تسمية. التسميات الجيدة تساعدك على إيجاد «قبل المعركة الأخيرة» بسرعة.

مخطط توضيحي

Git Bash (ليس لقطة حقيقية) · رسم توضيحي

```
$ git commit -m "Add homepage"
[main (root-commit) 3f2a9c1] Add homepage
 2 files changed, 20 insertions(+)
 create mode 100644 index.html
 create mode 100644 style.css
```

الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

```
git commit -m "Add homepage"
git commit -m "Fix navigation styling"
git commit -m "Update README"
```

الخيار: `-m` الرسالة. نفّذه بعد التجهيز فقط. الشكل 1 يعرض أول commit في المشروع.

أخطاء شائعة وحلولها

«Author identity unknown»	اضبط `user.name` و `user.email` (الدرس 2.3).
«nothing to commit»	لا شيء مجهّز. نفّذ `git add` أولًا.
نسيان علامتي الاقتباس.	ضع الرسالة بين علامتي اقتباس مزدوجتين.

تمرين عملي

احفظ `index.html` و `style.css` المجهّزين في commit برسالة `Add homepage` .

النتيجة المتوقعة

يطبع Git سطرًا يبدأ بـ `[main (root-commit)` ويسرد ملفين أُنشئا، كما في الشكل 1.

اختبر فهمك

1. ماذا يعني -m في git commit -m؟

- ○ نقل (Move)
- ○ رسالة (Message)
- ○ دمج (Merge)

2. أيّ هذه رسالة جيدة؟

- ○ Add homepage
- ○ stuff
- ○ asdf

3. أين يوجد الـ commit الجديد؟

- ○ لا مكان
- ○ على جهازك
- ○ على GitHub

الإجابات

- رسالة (Message) — يعطي رسالة الـ commit.
- Add homepage — تبدأ بفعل وتقول ما الذي تغيّر.
- على جهازك — الـ push هو الخطوة التي ترفعه.

الخلاصة

الـ commit يحفظ التغييرات المجهّزة مع رسالة. استخدم رسائل واضحة، وتذكّر أن الـ commits محلية حتى تنفّذ push.

مصطلحات أساسية

Commit hash	معرّف فريد لـ commit
root-commit	أول commit في المستودع
m-	خيار لإعطاء رسالة الـ commit

الدرس التالي

الدرس 4.3 — مراجعة سجل Git. صرت تحفظ اللقطات. بعده ستقرأها.

مراجعة سجل Git

Reviewing Git history

أهداف التعلم

- أن تقرأ قائمة الـ commits بـ `git log` .
- أن تقارن التغييرات بـ `git diff` و `git diff --staged` .
- أن تفحص commit بـ `git show` .

المتطلبات المسبقة

الدرس 4.2.

مقدمة

لا يفيد التاريخ إلا إذا استطعت قراءته. هذه الأوامر تعرض ما حدث ومتى.

الشرح

`git log` يسرد الـ commits من الأحدث إلى الأقدم. يعرض كل بند الـ hash الكامل والمؤلف والتاريخ والرسالة. وإن طالت القائمة فتح Git عارضًا: اضغط **q** للخروج منه.

`git log --oneline` يعرض سطرًا قصيرًا لكل commit: hash مختصر والرسالة. والعلامة `(HEAD → main)` تبيّن أين أنت الآن.

`git diff` يعرض التغييرات التي أجريتها لكنك **لم تجهّزها**. و `git diff --staged` يعرض التغييرات المجهّزة للـ commit القادم. الأسطر التي تبدأ بـ `+` أُضيفت، والتي تبدأ بـ `-` أُزيلت.

`git show` يعرض آخر commit مع تغييراته. و `git show 3f2a9c1` يعرض commit تختاره.

`git status` يبقى فحصك الأول: يخبرك بما تفعله بعد ذلك.

تشبيه من الحياة

> `git log` سجل الرحلة في السفينة. و `git diff` يقارن صفحة اليوم بما كُتب سابقًا.

مخطط توضيحي



الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

`git log`

`git log --oneline`

`git diff`

`git diff --staged`

`git show`

`git status`

الشكل 1 يعرض `git log --oneline` بعد ثلاثة commits.

أخطاء شائعة وحلولها

العالق داخل git log.	اضغط `q` للخروج من العارض.
توقع أن يعرض git diff التغييرات المجهّزة.	استخدم `git diff --staged` لها.
نسخ hash كامل يدويًا.	يكفي أول 7 أحرف.

تمرين عملي

عدّل `README.md` ونفّذ `git diff` . جهّزه ونفّذ `git diff --staged` . ثم نفّذ `git log --oneline` .

النتيجة المتوقعة

يعرض أول diff أسطرك المضافة بعلامة `+` . وبعد التجهيز لا يعرضها إلا `--staged` . ويسرد السجل commits التي أنشأتها.

اختبر فهمك

1. كيف تخرج من عارض git log؟

- ○ بكتابة exit()
- ○ بالضغط على q
- ○ بإغلاق ويندوز

2. أي أمر يعرض التغييرات غير المجهّزة؟

- ○ git diff
- ○ git log
- ○ git show

3. ماذا يعرض git log --oneline؟

- ○ كلمات سرك
- ○ سطرًا قصيرًا لكل commit
- ○ آخر ملف فقط

الإجابات

- بالضغط على q — الحرف q يغلق العارض.
- git diff — git diff يقارن ملفات عملك بمنطقة التجهيز.
- سطرًا قصيرًا لكل commit — hash مختصر والرسالة لكل commit.

الخلاصة

git log يسرد التاريخ، وgit diff وgit diff --staged يقارنان التغييرات، وgit show يفحص commit واحدًا.

مصطلحات أساسية

HEAD	الـ commit الذي أنت عليه الآن
git diff	عرض التغييرات غير المجهّزة
git show	عرض commit مع تغييراته

الدرس التالي

الوحدة 5 — ربط Git مع GitHub. تاريخك محلي. بعده ترسله إلى الإنترنت.

المختبر العملي 4 · أنشئ ثلاثة commits وافحصها

1. نفّذ `git add index.html style.css` ثم `git commit -m "Add homepage"`.
2. أضف السطر `p { line-height: 1.6; }` إلى `style.css`. افحص بـ `git diff` وجهّزه ثم `git commit -m "Improve text spacing"`.
3. أضف جملة واحدة إلى `README.md`. نفّذ `git add README.md` ثم `git commit -m "Update README"`.
4. نفّذ `git log --oneline` و `git show`.

الفحص	ينجح إذا...
ثلاثة commits	`git log --oneline` يعرض ثلاثة أسطر
الرسائل	كل رسالة تبدأ بفعل وتصف التغيير
التجهيز الانتقائي	كل commit يحتوي الملفات المقصودة فقط
حالة نظيفة	`git status` يقول «nothing to commit, working tree clean»

معيار التقييم: 4 فحوص من 4 = نجاح.

اختبار الوحدة 4 (علامة النجاح: 70%)

1. أي أمر يجهّز ملفًا واحدًا؟

- ○ git push index.html
- ○ git log index.html
- ○ git add index.html

2. ماذا يفعل git add ؟.

- ○ يرفع إلى GitHub
- ○ يجهّز التغييرات في المجلد الحالي
- ○ ينشئ commit

3. أي أمر يحفظ التغييرات المجهّزة؟

- ○ "git commit -m "message
- ○ git add -p
- ○ git diff

4. كيف ترى التغييرات المجهّزة؟

- ○ git init
- ○ git diff --staged
- ○ git diff

5. ماذا تعني (HEAD -> main) في السجل؟

- ○ الملف محذوف
- ○ الـ commit على GitHub
- ○ أنت على أحدث commit في main

الإجابات

- git add index.html — git add متبوعًا باسم الملف يجهّزه.
- يجهّز التغييرات في المجلد الحالي — يجهّز ولا ينشئ commit ولا يرفع.
- "git commit -m "message — الـ commit ينشئ اللقطة.
- git diff --staged — الخيار --staged يقارن منطقة التجهيز بآخر commit.
- أنت على أحدث commit في HEAD — main يحدد موقعك الحالي.